

Una zona tua, risposta sul filo

BIND9 che serve una zona diretta e una inversa, e un client che risolve attraverso di lui sulla LAN del laboratorio. Ogni blocco qui sotto è stato catturato a lab acceso — compresa la query stessa, vista partire dal client e arrivare al server.

Comando `qlab run dns-lab`

VM `dns-lab-server` 10.20.30.1 · `dns-lab-client` 10.20.30.50

Zone `lab.qlab` diretta · `30.20.10.in-addr.arpa` inversa

Esito `qlab test dns-lab` → 7 esercizi, tutti superati

1. Le due macchine, e l'indirizzo che non è finto

```
--- server ---
ens3          UP          10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4          UP          10.20.30.1/24 fe80::5054:ff:fe00:e01/64
--- client ---
ens3          UP          10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4          UP          10.20.30.50/24 fe80::5054:ff:fe00:e02/64
```

La LAN del laboratorio è `10.20.30.0/24`, e non è una scelta arbitraria: è la sottorete che il file di zona stesso dichiara. `ns1.lab.qlab` è definito come `10.20.30.1`, e il server risponde davvero su `10.20.30.1`. Il record è quindi vero e non decorativo — si può risolvere `ns1.lab.qlab` e poi pingare il risultato, raggiungendo la macchina che ha dato la risposta.

Ogni VM porta anche la scheda SLIRP di QEMU per `qlab shell`. Due interfacce su una macchina DNS sono esattamente la situazione in cui «su quale indirizzo sta ascoltando il mio server» smette di essere una domanda retorica.

2. Di cosa il server è autoritativo

```
zone "lab.qlab" {
    type master;
    file "/etc/bind/zones/db.lab.qlab";
};

zone "30.20.10.in-addr.arpa" {
    type master;
    file "/etc/bind/zones/db.10.20.30";
};
```

Due zone: dai nomi agli indirizzi, e dagli indirizzi ai nomi. Sono file separati con numeri di serie separati, e a tenerli d'accordo non c'è nulla se non voi — per questo su reti vere una risoluzione inversa così spesso contraddice quella diretta.

```
$TTL      604800
@         IN      SOA      ns1.lab.qlab. admin.lab.qlab. (
                        2024010101 ; Serial
                        3600       ; Refresh
                        1800       ; Retry
                        604800     ; Expire
                        86400 )    ; Negative Cache TTL
@         IN      NS       ns1.lab.qlab.
ns1       IN      A        10.20.30.1
web       IN      A        10.20.30.10
mail      IN      A        10.20.30.20
db        IN      A        10.20.30.30
app       IN      A        10.20.30.40
mail2     IN      A        10.20.30.21
web       IN      AAAA     2001:db8::10
www       IN      CNAME    web.lab.qlab.
ftp       IN      CNAME    web.lab.qlab.
@         IN      MX       10 mail.lab.qlab.
```

```
@      IN      MX      20 mail2.lab.qlab.
@      IN      TXT      "v=spf1 mx -all"
_dmarc IN      TXT      "v=DMARC1; p=reject"
_http._tcp IN      SRV      10 0 80 web.lab.qlab.
_mysql._tcp IN      SRV      10 0 3306 db.lab.qlab.
```

Tutto quello che un principiante incontra, in un posto solo: il `SOA` con i suoi tempi, `NS`, `A` e `AAAA`, gli alias `CNAME`, gli `MX` con le priorità, i `TXT` che trasportano le policy SPF e DMARC, e gli `SRV` che pubblicizzano un servizio con la sua porta.

```
active
named-checkconf: OK
zone lab.qlab/IN: loaded serial 2024010101
OK
```

`named-checkconf` e `named-checkzone` sono i due comandi da ricordare: BIND si rifiuta di caricare una zona con un errore di sintassi e continua a servire la precedente, quindi «ho modificato il file e non è cambiato niente» è quasi sempre una zona che non è mai stata caricata.

3. Una query che attraversa la rete

Il client chiede; `tcpdump` sul server la guarda arrivare.

```
--- on the client ---
web.lab.qlab.      604800 IN  A   10.20.30.10
web.lab.qlab.

--- on the server: tcpdump -i any -n port 53 ---
09:23:08.216709 IP 10.20.30.50.53658 > 10.20.30.1.53: 12135+ [1au] A? web.lab.qlab. (53)
09:23:08.216932 IP 10.20.30.1.53 > 10.20.30.50.53658: 12135* 1/0/1 A 10.20.30.10 (85)
09:23:08.217088 IP 10.20.30.1.53 > 10.20.30.50.53658: 12135* 1/0/1 A 10.20.30.10 (85)
09:23:08.236666 IP 10.20.30.50.34485 > 10.20.30.1.53: 4534+ [1au] PTR? 10.30.20.10.in-addr.arpa. (65)
```

La cattura è tutta la lezione. Una query parte da `10.20.30.50` da una porta alta casuale, arriva su `10.20.30.1` porta 53, e la risposta torna indietro subito. Il `+` dopo l'id della query significa che è stata chiesta ricorsione; l' `*` nella risposta significa che il server è **autoritativo** per quel nome — non è andato a chiedere a nessun altro, semplicemente lo sa.

L'ultima riga è la risoluzione inversa, e mostra il trucco di `in-addr.arpa`: per chiedere di `10.20.30.10` si interroga il nome `10.30.20.10.in-addr.arpa`, con gli ottetti rovesciati, perché il DNS delega da destra mentre gli indirizzi si scrivono partendo dalla parte più significativa.

4. I tipi di record, visti dal client

```
A      web.lab.qlab      10.20.30.10
AAAA   web.lab.qlab      2001:db8::10
CNAME   www.lab.qlab      web.lab.qlab.
MX      lab.qlab          20 mail2.lab.qlab. 10 mail.lab.qlab.
TXT     lab.qlab          "v=spf1 mx -all"
SRV     _http._tcp.lab.qlab 10 0 80 web.lab.qlab.
NS      lab.qlab          ns1.lab.qlab.
```

`MX` restituisce due host con priorità — vince il numero più basso, e `mail2` a 20 è la riserva. `SRV` porta con sé una porta, ed è così che un client trova un servizio senza che nessuno abbia scritto `:80` da qualche parte. `CNAME` restituisce un altro *nome*, non un indirizzo, quindi il resolver deve fare un altro giro.

5. Il dominio di ricerca, che si comporta diversamente in ogni strumento

```
$ dig www +short
(empty: dig ignores the search list)
$ dig +search www +short
web.lab qlab.
10.20.30.10
$ host www
www.lab.qlab is an alias for web.lab.qlab.
web.lab.qlab has address 10.20.30.10
web.lab.qlab has IPv6 address 2001:db8::10
```

Qui ci si inciampa di continuo. `dig` ignora `search` di `/etc/resolv.conf` se non gli si passa `+search`; `host` e `nslookup` lo applicano. Quindi `dig www` che non restituisce niente mentre `host www` funziona non è un server rotto — sono due strumenti con impostazioni predefinite diverse.

Come risolve il client

```
nameserver 127.0.0.1
search lab.qlab

# Forward every query to the lab's authoritative server, straight over
# the lab LAN. (This used to point at 10.0.2.2#5354 – the host, on a
# hardcoded port – but QLab allocates host ports dynamically, so that
# number was almost never the right one and resolution simply hung.)
no-resolv
server=10.20.30.1
# Add lab.qlab to search domain
domain=lab.qlab
```

Il client punta a sé stesso, e un `dnsmasq` locale inoltra tutto a `10.20.30.1`. È una piccola cache davanti al server autoritativo, ed è come sono organizzate quasi tutte le reti vere.

6. Verifica

```
Exercise 1: All 5 checks passed
Exercise 2: All 6 checks passed
Exercise 3: All 2 checks passed
Exercise 4: All 2 checks passed
Exercise 5: All 3 checks passed
Exercise 6: All 3 checks passed
Exercise 7: All 13 checks passed
Exercises run:      7
```

```
Exercises passed: 7
Exercises failed: 0
Exercises skipped: 0
All exercises passed!
```

L'esercizio 7 è quello che controlla la rete: gli esercizi da 1 a 6 interrogano il server *dal server* con `@localhost`, e passerebbero anche se le due macchine non riuscissero affatto a raggiungersi.

7. Cosa portarsi via

- Diretta e inversa sono due zone indipendenti. Non c'è niente che le sincronizzi.
- Una risposta autoritativa (`*` in una cattura, `aa` in un `dig` completo) significa che il server sa, non che ha chiesto a monte. È la differenza fra servire una zona ed essere un resolver.
- `in-addr.arpa` rovescia gli ottetti perché il DNS delega da destra.
- `dig` non usa la search list se non glielo si dice. `host` sì.
- Se una modifica sembra non avere effetto, si lanci `named-checkzone`: BIND continua a servire l'ultima zona caricata senza errori.

`guide.md` del plugin va oltre: aggiungere record e incrementare il seriale, la delega inversa, e leggere una catena di risoluzione con `+trace`.